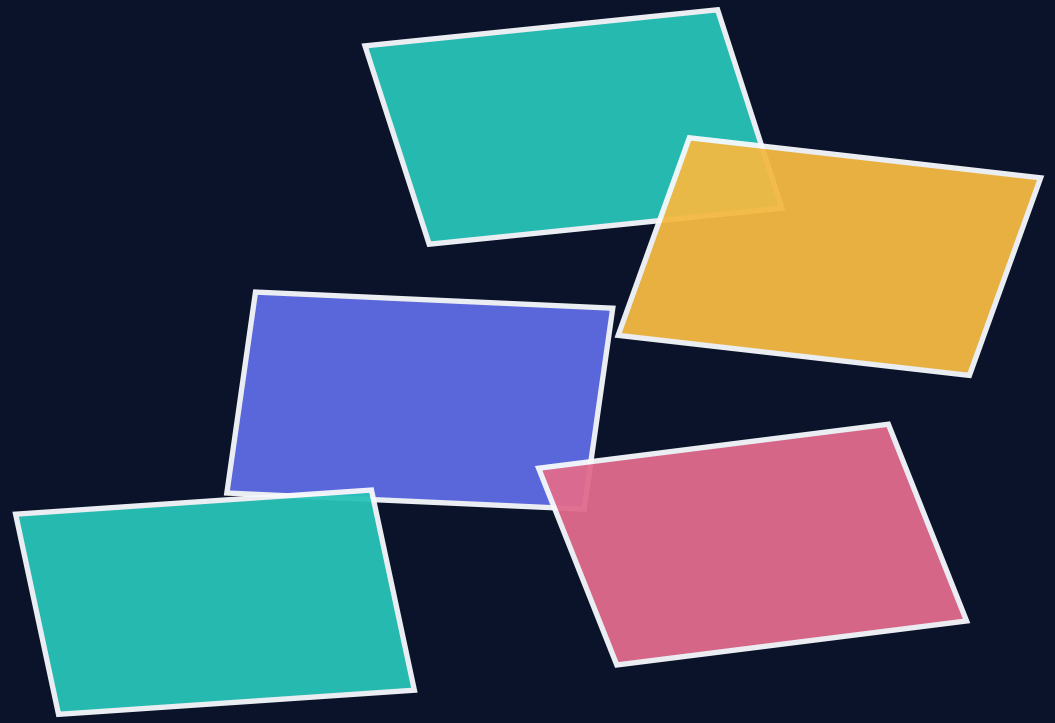
# PACKING SQUARES, PROFESSIONALLY.

A validation-first optimization pipeline for Kaggle

From geometry constraints to a reproducible submission.



Python · SAT geometry · simulated annealing · CI · Kaggle

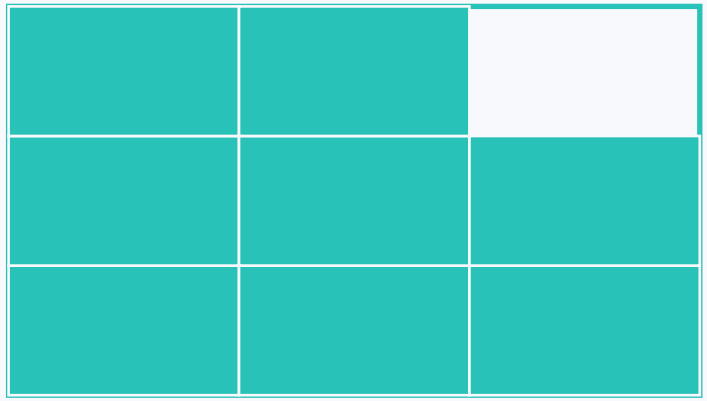
# The problem: pack more, waste less

**For each puzzle n, place n unit squares inside the smallest axis-aligned square.**

Each square is described by a center (x, y) and a rotation angle θ.
The evaluator recomputes the enclosing side length L.

$$\text{score}(n) = L / \sqrt{n} - 1$$

Lower is better · leaderboard sums all puzzles



A 3 × 3 container for 8 squares

# A three-layer solver architecture

## The professional move was to separate correctness from search.

Every optimization experiment must pass through the same geometry and delivery gates.

### GEOMETRY

Vertices
SAT overlap
Boundary checks

### SOLVERS

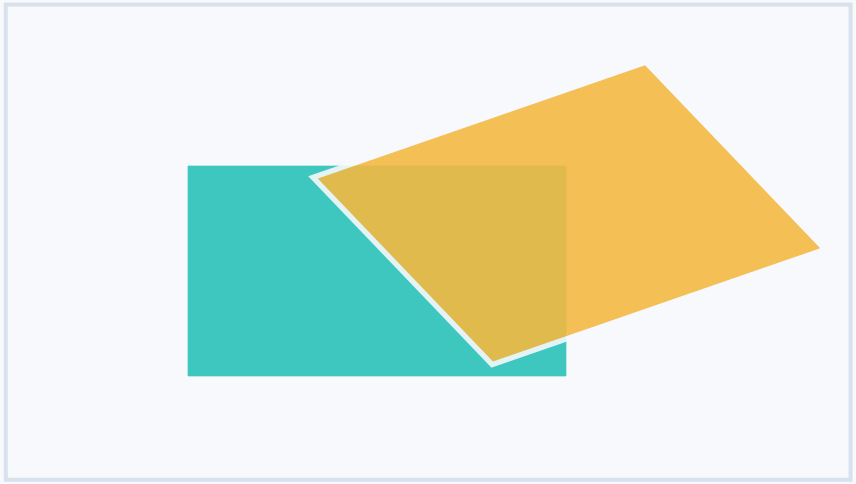
Grid baseline
Rectangular layouts
Annealing

### DELIVERY

Streaming CSV
Schema validator
Kaggle + GitHub

# Constraint engineering: prove validity

**Rotated squares are convex polygons—so the Separating Axis Theorem gives a clean test.**



Positive-area overlap → reject
Edge/corner touch → allowed

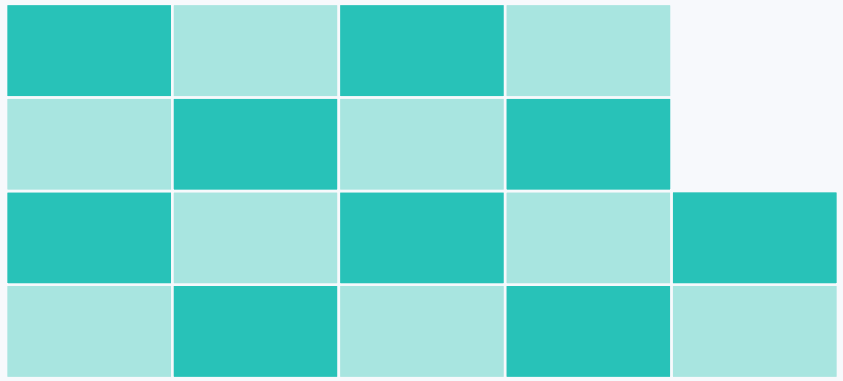
1. **Compute vertices from (x, y, θ)**
2. **Project both polygons on each edge normal**
3. **A separating axis proves non-overlap**

Boundary checks use the actual rotated vertices, not just centers.

# Start with a strong, safe baseline

## A complete baseline is more valuable than an incomplete clever idea.

For each n, choose the best near-square integer rectangle and place one square per cell.



18 squares → 5 × 4 container

**1,000**

puzzles covered

**500,500**

data rows

**0°**

rotation in fallback

The baseline is deterministic, cheap, explainable, and always available whe

# Improve selectively with annealing

**Simulated annealing explores continuous centers and angles without sacrificing the fallback.**




**START**

rectangular layout

**PERTURB**

move one square / rotate

**SCORE**

container side + penalties

**KEEP**

accept improvement or escape local minima

**Engineering rule: optimize small puzzle sizes first; retain the validated deterministic layout for everything else.**

# What shipped

**The pipeline produced a complete, validated, and submitted artifact.**

**500,500**
submission rows

**1,000**
puzzles

**28.9377**
local baseline score

**VALID**
geometry gate

**Kaggle accepted submission.csv and queued it for evaluation.**

# Lessons for practical optimization

01 **Separate correctness from search.**

02 **Make the baseline complete and reproducible.**

03 **Use exact geometry where invalidity is expensive.**

04 **Deliver the whole workflow: tests, CI, artifacts, and provenance.**

Next step: replace the baseline with stronger search while preserving the same validation gates.